

Untersuchung des CG-Verfahrens für die Lösung des Poisson-Problems

Alexander Huhn, Fedir Deineko

2. Februar 2026

Inhaltsverzeichnis

1	Einführung und Motivation	2
2	Theoretische Grundlagen	2
2.1	Zweite finite Differenz	2
2.2	Poisson-Problem	3
2.3	Definition der Block-Diagonal-Matrizen	3
2.4	Aufstellen des linearen Gleichungssystems	3
2.5	Approximationsfehler	3
2.6	Speicherplatzbedarf der Matrix A	3
2.7	LU-Zerlegung	4
2.8	Conjugate-Gradient-Verfahren	4
2.9	Konvergenz und Konvergenzrate	5
2.9.1	Kondition von der Matrix A	6
3	Experimente	6
3.1	Motivation	6
3.2	Setting	6
3.3	Die exakte und approximierte Lösung	6
3.4	Konvergenz des CG-Verfahrens	8
3.5	Approximationsfehler vom CG-Verfahren mit unterschiedlichem ε .	9
3.6	Approximationsfehler vom CG-Verfahren für ein festes ε vs $\varepsilon = h^4$	10
3.7	Approximationsfehler LU vs CG	11
3.8	Laufzeit CG-Verfahren vs LU-Zerlegung	12
4	Zusammenfassung	13
5	Python-Dokumentation	14
5.1	Overview	14
5.2	experiments_it.py	14
5.2.1	Purpose	14
5.2.2	Functions	15

5.3	block_matrix_2d.py	15
5.3.1	Purpose	15
5.3.2	Class: <code>BlockMatrix</code>	16
5.3.3	Additional Function	17
5.4	poisson_problem_2d.py	18
5.4.1	Purpose	18
5.4.2	Funktionen	18
5.5	linear_solvers.py	21
5.5.1	Purpose	21
5.5.2	Implemented	21
5.5.3	Placeholders	22
5.6	time_comp.py	22
5.6.1	Purpose	22
5.7	User manual	23
6	Quellenverzeichnis	26

1 Einführung und Motivation

Ziel dieser Arbeit ist die numerische Lösung großer linearer Gleichungssysteme, wie sie bei der Diskretisierung des Poisson-Problems entstehen, mithilfe iterativer Verfahren. Dabei werden iterative und direkte Verfahren systematisch hinsichtlich Rechenaufwand, Speicherplatzbedarf und Qualität der numerischen Lösung verglichen. Ein besonderer Fokus liegt dabei auf dem Einfluss verschiedener Abbruchbedingungen auf das Konvergenzverhalten iterativer Algorithmen. In unserer Arbeit haben wir uns für das CG-Verfahren entschieden. Dieses Verfahren werden wir dann auch letztendlich mit der LU-Zerlegung vergleichen.

2 Theoretische Grundlagen

2.1 Zweite finite Differenz

Definition 2.1 (Zweite finite Differenz). Sei $h \in \mathbb{R}$ mit $0 < h$, $x \in (a, b)$, $x + h \in [a, b]$ und $f : [a, b] \rightarrow \mathbb{R}$. Dann ist:

$$D_h^{(2)} f(x) := \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

die **zweite finite Differenz**.

Satz 2.2. Sei $0 < h \in \mathbb{R}$, $x \in (a, b)$, $x + h \in [a, b]$ und $f : [a, b] \rightarrow \mathbb{R}$ mit $f \in C^\infty([a, b])$. Dann gilt:

$$D_h^{(2)} f(x) = f''(x) + \mathcal{O}(h^2)$$

2.2 Poisson-Problem

Das Poisson-Problem haben wir ausführlich im zweiten Projekt behandelt, es lautet $-\Delta u(x) = f(x)$. Die Differenzialgleichung wird dabei diskretisiert und mithilfe von der zweiten finiten Differenz approximiert. Daraus entstehen große lineare Gleichungssysteme.[6]

2.3 Definition der Block-Diagonal-Matrizen

Sei $N = (n - 1)^2$. Die Matrix $A \in \mathbb{R}^{N \times N}$, die beim 2D-Finite-Differenzen-Ansatz entsteht, besteht aus Blockmatrizen.

Die Matrix $C \in \mathbb{R}^{(n-1) \times (n-1)}$ führt zur Gesamtmatrix $A \in \mathbb{R}^{N \times N}$:

$$C := \begin{pmatrix} 4 & -1 & 0 & 0 & \cdots & 0 \\ -1 & 4 & -1 & 0 & \cdots & 0 \\ 0 & \ddots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & -1 & 4 & -1 \\ 0 & \cdots & 0 & 0 & -1 & 4 \end{pmatrix} \quad A := \begin{pmatrix} C & -I & 0 & \cdots & 0 \\ -I & C & -I & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & -I & C & -I \\ 0 & \cdots & 0 & -I & C \end{pmatrix}$$

Die Schrittweite lautet $h = 1/n$. Da A sehr dünn besetzt ist, lohnt sich die Speicherung als sparse Matrix.

2.4 Aufstellen des linearen Gleichungssystems

Die diskrete Gleichung lautet:

$$-h^{-2}Au(x) = f(x).$$

Durch die beschriebene Approximation des Laplace-Operators ergeben sich N lineare Gleichungen.

2.5 Approximationsfehler

Definition 2.3. Sei $\hat{u}$ die approximierte Lösung und u die exakte Funktion und u_i zu $\hat{u}_i$ der dazugehörige Funktionswert am gleichen Diskretisierungspunkt. Wir definieren den **Approximationsfehler** dann als:

$$e_u^{(n)} = \max_{i=1, \dots, (n-1)^2} |\hat{u}_i - u_i|$$

2.6 Speicherplatzbedarf der Matrix A

Wir untersuchen den Speicherplatzbedarf der Matrix A im *sparse* Format und vergleichen ihn mit einer vollbesetzten Matrix. Im *sparse* Format kann man di-, bi- und triagonale Matrizen, aber auch Matrizen mit Diagonalen links und rechts

der Hauptdiagonale effizient speichern. Das Format speichert nur die Werte auf diesen Diagonalen.

In unserem Fall können wir die Matrix A zusammenstellen aus den $-I$ und C Matrizen. $-I$ lässt sich als diagonale Matrix im *sparse* Format darstellen lassen, man speichert nur die Werte auf der Diagonalen und C speichern wir als eine tridiagonale Matrix auch im *sparse* Format.

Definition 2.4. Die Anzahl der nicht-Nullelemente einer Matrix M wird mit $\#(M)$ bezeichnet.

Die Anzahl der nicht-Nullelemente der Matrix I ist gleich $\#(I) = (n - 1)$ und $\#(C) = (n - 1) + 2(n - 2)$. Daraus folgt, $\#(A) = (n - 1)\#(C) + 2(n - 2)\#(I) = 5(n - 1)^2 - 4(n - 1) = 5N - 4\sqrt{N}$.

Wie bereits in Arbeitsblatt 2 ist die Matrix A dünnbesetzt (die Matrix A hat viele Einträge gleich Null). Außerdem ist die Matrix A symmetrisch und positiv definit (notwendige Bedingung für das CG-Verfahren).

Definition 2.5. Iterative Verfahren sind mathematische Techniken, die zur Lösung von Gleichungen oder Optimierungsproblemen eingesetzt werden, indem sie eine Folge von Näherungslösungen schrittweise verbessern. Diese Methoden sind besonders nützlich, wenn direkte Lösungsansätze zu komplex sind.

2.7 LU-Zerlegung

Definition 2.6. Sei $A, P, L, U \in \mathbb{R}^{m \times m}$ und P sei eine Permutationsmatrix, L eine untere Dreiecksmatrix und U eine obere. Dann ist

$$A = PLU$$

eine LU-Zerlegung der Matrix A .

Satz 2.7. Jede Matrix $A \in \mathbb{R}^{m \times m}$ hat eine LU-Zerlegung.[2]

2.8 Conjugate-Gradient-Verfahren

Voraussetzungen: A soll symmetrisch und positiv definit sein. Die Methode minimiert iterativ die Funktion

$$\Phi(x) = \frac{1}{2} \langle Ax, x \rangle - \langle b, x \rangle$$

durch folgende Schritte:

Initialisierung: Wähle $x_0 \in \mathbb{R}^m$, $\varepsilon \in (0, 1)$, $k = 0$, $r_0 = b - Ax_0$, $d_0 = r_0$, $\varepsilon = 10^{-8}$

Iteration über k : Berechne $z_k = Ad_k$,

$$\alpha_k = \frac{\|r_k\|}{\langle z_k, d_k \rangle}, \quad r_{k+1} = r_k - \alpha_k z_k, \quad \beta_k = \frac{\|r_{k+1}\|^2}{\|r_k\|^2},$$

$$x_{k+1} = x_k + \alpha_k d_k, \quad d_{k+1} = r_{k+1} + \beta_k d_k.$$

Falls $\|r_k\| \leq \varepsilon \|r_0\|$ Abbruch.[2]

Satz 2.8. Beim CG-Verfahren ist $r_k = b - Ax_k$ [2]

Das CG-Verfahren findet die genaue Lösung in genau N Schritten. Wir können dieses Verfahren zur Lösung des aufgestellten LGS durch die Diskretisierung des Poisson-Problems verwenden, da die Matrix A strikt diagonaldominant und symmetrisch ist, denn aus dem Greshgori-Kreise-Satz folgt, dass die Eigenwerte positiv sind und daraus auch, dass sie positiv definit ist.

CG-Verfahren vs. LU-Zerlegung

Im Gegensatz zu direkten Verfahren wie der LU-Zerlegung, die das System durch endliche Elimination lösen, ist das CG-Verfahren ein Iterationsverfahren. Theoretisch bricht das CG-Verfahren in exakter Arithmetik nach maximal $N = (n - 1)^2$ Schritten ab und liefert die exakte Lösung des diskreten Systems. In der Praxis wird jedoch weit früher abgebrochen, sobald die gewünschte Toleranz ε unterschritten ist. Ein entscheidender Nachteil der LU-Zerlegung für das Poisson-Problem ist der sogenannte Fill-in-Effekt. Obwohl die Systemmatrix A dünnbesetzt ist, sind die resultierenden Dreiecksmatrizen L und U deutlich dichter besetzt.

Die Wahl von ε ist kritisch für die Effizienz. Da die Diskretisierung bereits einen Fehler der Ordnung $\mathcal{O}(h^2)$ aufweist (siehe Satz 2.2), ist es numerisch nicht sinnvoll, die Iteration bis zur Maschinengenauigkeit fortzuführen.

Das Ziel ist es, ε so zu wählen, dass der Iterationsfehler klein gegenüber dem Approximationsfehler $e_u^{(n)}$ ist, um Rechenzeit zu sparen, ohne die Qualität der Lösung zu verschlechtern.

2.9 Konvergenz und Konvergenzrate

Die Genauigkeit der numerischen Lösung des Poisson-Problems wird maßgeblich durch die Konditionszahl $\kappa(A)$ der Systemmatrix beeinflusst. Die mit dem Conjugate-Gradient-Verfahren (CG) gelöste Poisson-Aufgabe konvergiert linear, wobei die Konvergenzgeschwindigkeit durch die spektralen Eigenschaften der Matrix A bestimmt wird.

$$\|x_k - x^*\|_A \leq 2 \left(\frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1} \right)^k \|x_0 - x^*\|_A,$$

wobei [2]

- x_k ist die Näherung in der k -ten Iteration,
- x^* ist die exakte Lösung,
- $\kappa(A) = \frac{\lambda_{\max}}{\lambda_{\min}}$ ist die Konditionszahl der Matrix A .

2.9.1 Kondition von der Matrix A

Für die finiten Differenzen auf einem $n \times n$ Gitter mit Gitterweite $h = \frac{1}{n}$ sind die Eigenwerte der Matrix A analytisch bekannt:

$$\lambda_{j,k} = \frac{4}{h^2} \left(\sin^2 \left(\frac{j\pi h}{2} \right) + \sin^2 \left(\frac{k\pi h}{2} \right) \right), \quad j, k = 1, \dots, n$$

Daraus ergibt sich für die spektrale Konditionszahl ($\kappa_2(A) = \frac{\lambda_{\max}}{\lambda_{\min}}$):

$$\kappa(A) \approx \frac{8/h^2}{2\pi^2} = \frac{4}{\pi^2 h^2} = O(h^{-2}) \quad (1)$$

Da die Gesamtanzahl der inneren Gitterpunkte $N = n^2 \approx h^{-2}$ ist, wächst die Konditionszahl linear mit der Problemgröße: $\kappa(A) \approx O(N)$. [7]

3 Experimente

3.1 Motivation

In diesem Projekt untersuchen wir das CG-Verfahren für die Lösung des Poisson-Problems und vergleichen es mit der LU-Zerlegung. Beim CG-Verfahren können wir anders als bei der LU-Zerlegung das sparse-Format der Matrix A ausnutzen, weil beim CG-Verfahren multipliziert man iterativ die Matrix A und es werden dabei keine Matrizen verändert oder manipuliert. Das sollte theoretisch bessere Ergebnisse mit Hilfe des CG-Verfahrens bringen. Deshalb betrachten wir den Approximationsfehler und die Laufzeit der unterschiedlichen Verfahren. Beim CG-Verfahren untersuchen wir auch noch den Fehler, wenn man beim Abbruchkriterium $\|r_k\| \leq \varepsilon \|r_0\|$ das Epsilon verändert.

3.2 Setting

Wir untersuchen wieder die Funktion wie im letzten Projekt $u : \Omega \rightarrow \mathbb{R}$ mit $u(x) = x_1 x_2 \sin(\kappa \pi x_1) \sin(\kappa \pi x_2)$. Wir setzen $\kappa = 1$. $f(x)$ ist gegeben durch

$$\begin{aligned} f(x) &= -\Delta u = \frac{\partial^2 u}{\partial x_1^2} + \frac{\partial^2 u}{\partial x_2^2} \\ &= 2x_2 \sin(x_2) \cos(x_1) - 2x_1 x_2 \sin(x_2) \sin(x_1) + 2x_1 \sin(x_1) \cos(x_2) \end{aligned}$$

Für das CG-Verfahren wählen wir das Abbruchkriterium **max_iter** = 10^{14} , da wir das Programm weiterlaufen wollen unabhängig von der Anzahl der Iterationen. Weiterhin bleibt **var_x** = 10^{-16} , da es keinen Sinn ergibt kleiner als die Computergenauigkeit zu rechnen.

3.3 Die exakte und approximierte Lösung

Wir stellen die exakte und approximierte Lösung für $n = 50$ und $\varepsilon = 10^{-11}$ dar.

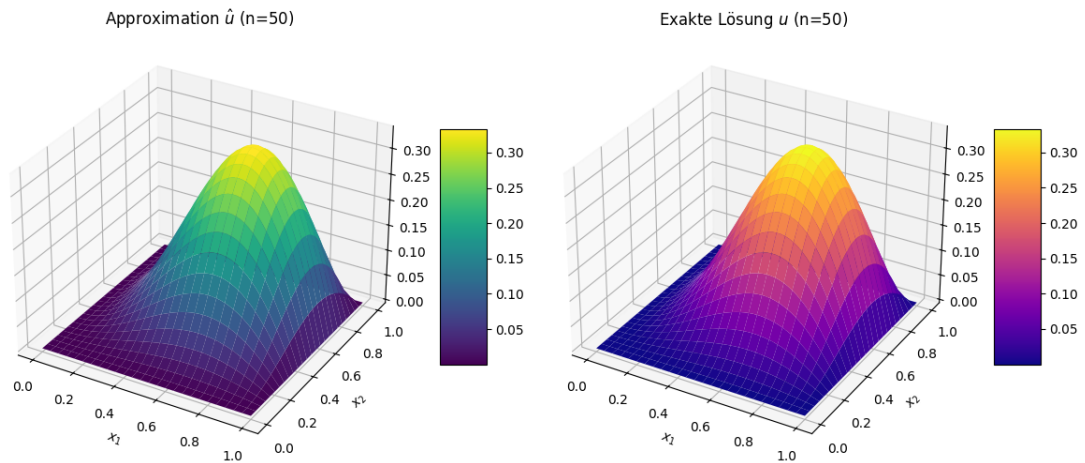


Abbildung 1: Darstellung der approximierten und genauen Lösung

Beobachtung

Die beiden Grafen in Abbildung 1 kann man mit den Augen nicht von einander unterscheiden, sie sind fast identisch. In Abbildung 2 sieht man die Differenz der beiden Grafen und sie sieht erstaunlich ähnlich wie die beiden Grafen von der Form her aus, nur ist Z-Achse viel kleiner skaliert. Dabei befindet sich der Fehler ungefähr in der gleichen Größenordnung. Abbruchkriterium: **eps**

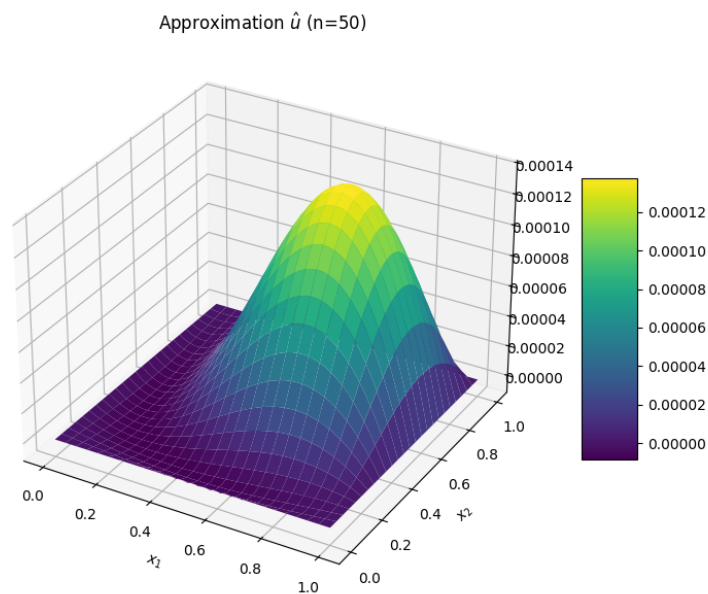


Abbildung 2: Darstellung der Differenz der approximierten und genauen Lösung

Auswertung

Die exakte und die approximierte Lösung kann man kaum unterscheiden, weil der Fehler ziemlich klein ist (ungefähr 10^{-4}). Den Grafen der Differenz kann man auf die Ergebnisse aus dem ersten Projekt zurückführen, denn dort haben wir gezeigt, dass bei den Extremstellen der Funktion bei der zweiten finiten Differenz die größten Abweichungen gab.[5]

3.4 Konvergenz des CG-Verfahrens

Wir untersuchen in diesem Experiment die Konvergenz des CG-Verfahrens für $n = 50$ und ein Mal $\varepsilon = 10^{-11}$ (blau) und $\varepsilon = h^4$ (schwarz). Die Konvergenz des Verfahrens zur Lösung ist nämlich ein wichtiger Teil der Laufzeit.

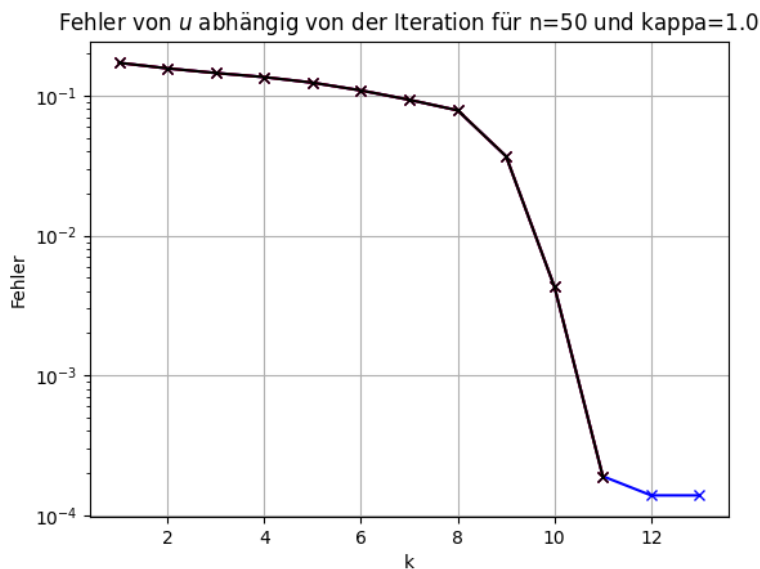


Abbildung 3: Konvergenz des CG-Verfahrens für $n = 50$

Beobachtung

In Abbildung 3 sieht man den Verlauf des Fehlers abhängig von den Iterationschritten k . Die zwei Grafen liegen über einander und unterscheiden sich fast nicht, außer dass der schwarze Graf vor dem blauen aufhört. Zuerst fällt der Fehler nur leicht, ab einem Zeitpunkt fällt er abrupt ab und flacht aus.

Abbruchkriterium: Beide Male **eps**

Auswertung

Wir wissen, dass im Worst-Case-Szenario der Fehler mit ungefähr $2 \left(\frac{\sqrt{\kappa(A)}-1}{\sqrt{\kappa(A)}+1} \right)^k$ abfällt und es widerspricht nicht der Grafik, da man zuerst in ungünstige Richtungen läuft bis man auf einmal stark voran kommt. Der schwarze Graf bricht vor dem blauen ab, aus dem Grund, dass der schwarze eine größeres Abbruchkriterium hat.

3.5 Approximationsfehler vom CG-Verfahren mit unterschiedlichem ε

Wir wollen den Approximationsfehler für unterschiedliche ε abhängig von $h = 1/n$ untersuchen. Dafür setzen wir $\varepsilon = h^k$ für $k \in \{-2, 0, 2, 4, 6\}$ ein. n geht so weit wie möglich bis zu $n = 1000$.

Beobachtung

In dieser Grafik sieht man den Verlauf des Approximationsfehlers der Lösung mithilfe des CG-Verfahrens für unterschiedliche ε . Der Fehler bei $k = -2$ bleibt ungefähr gleich bei 0,1 und steigt ein bisschen. Bei $k = 0$ fällt der Fehler stufenförmig ab, dennoch kann man keine sichere Konvergenz beobachten. Bei $k \in \{2, 4, 6\}$ konvergiert der Fehler mit $\mathcal{O}(h^2)$, denn man sieht dass alle drei Fehlergrafen sich überlappen und parallel zu $\mathcal{O}(h^2)$ laufen. Leider wird nicht die Abbruchstelle erreicht, bei der man sehen sollte, dass der Fehler nicht weiter konvergiert, sondern ansteigt.

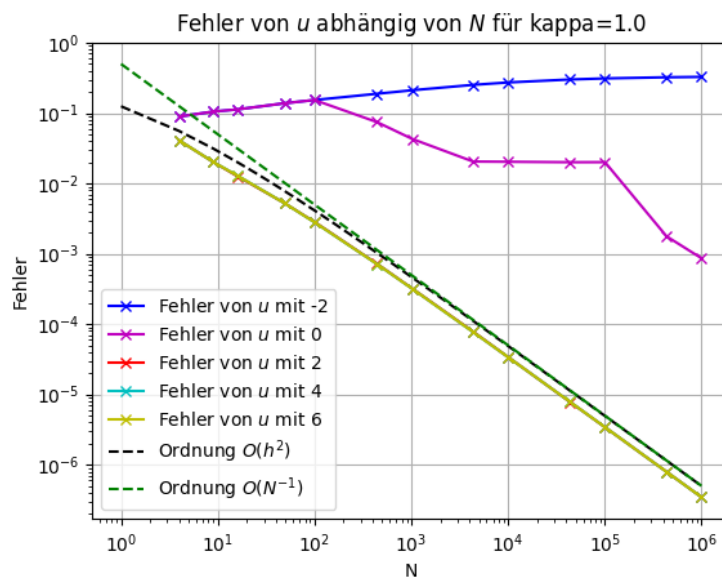


Abbildung 4: Fehler von u bezüglich ε

Auswertung

Bei $k = -2$ ist es klar, dass der Fehler nicht nach unten geht, weil der Algorithmus fast sofort abbricht, meistens gleich nach der ersten Iteration. Wenn $k = 1$ ist, dann kann es sein dass es ein Paar Iterationen gibt bis der Abbruch kommt. Ob der Fehler wirklich konvergiert oder es nur ein Zufall ist, ist nicht wirklich klar. Dass wenn $k \in \{2, 4, 6\}$ ist und sich die Fehlergrafen sich nicht unterscheiden, kann man damit erklären, dass das Abbruchkriterium nur die Laufzeit ersparen und nicht den Approximationsfehler verbessern kann. Aus diesem Grund gibt es beim CG-Verfahren keinen großen Unterschied, ob man das Abbruchkriterium mit

dem Approximationsfehler verkleinern lässt oder von Anfang an klein lässt.

3.6 Approximationsfehler vom CG-Verfahren für ein festes ε vs $\varepsilon = h^4$

Wir untersuchen den Approximationsfehler einzeln für ein maximales $n = 7072$ für $\varepsilon = 10^{-11}$ und $\varepsilon = h^4$.

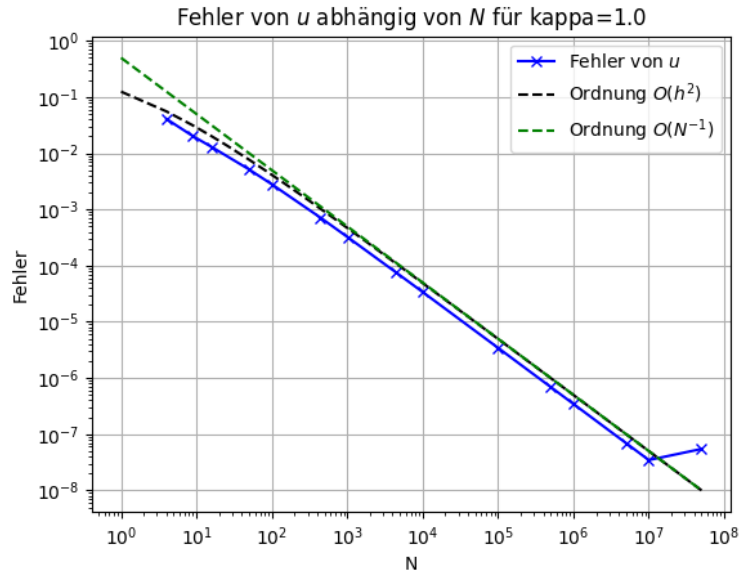


Abbildung 5: Fehler von u

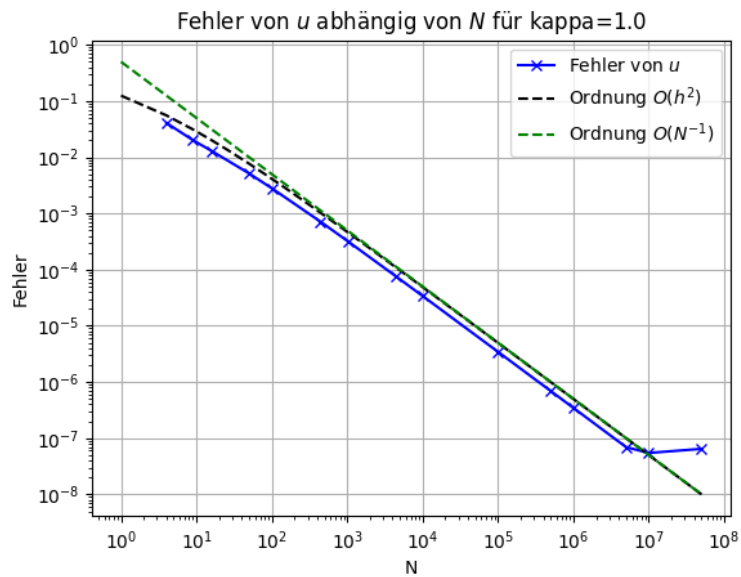


Abbildung 6: Fehler von u mit $\varepsilon = h^4$

Beobachtung

Die beiden Grafen sehen ziemlich ähnlich aus, es gibt kaum Unterschiede. Der Fehler konvergiert mit $\mathcal{O}(N^{-1})$ und bricht ab bei ungefähr $N = 10^7$.

Abbruchkriterium: **eps**

Auswertung

Der Fehler nimmt stetig ab, wenn N erhöht wird, was zeigt, dass feinere Gitter genauere Lösungen liefern. Bei sehr großen N (10^7 – 10^8) ist eine leichte Abflachung der Fehlerkurve zu beobachten, ausgelöst durch den steigenden Approximationsfehler ab einer Ordnung von 10^7 bis 10^8 . Die Plots liefern eine visuelle Bestätigung der Konvergenz zweiter Ordnung, was mit der theoretischen Erwartung der finiten Differenzen übereinstimmt. Das Ansteigen des Fehlers am rechten Rand ist kein Fehler in dem Algorithmus, sondern ein physikalisches Limit der Computerhardware:

- Rundungsfehler: Bei sehr feinen Gittern (N sehr groß) werden die Abstände h so klein, dass die Differenzen im Laplace-Operator ($\frac{u_{i+1}-2u_i+u_{i-1}}{h^2}$) numerisch instabil werden.
- Kondition der Matrix: Mit steigendem N wird die Systemmatrix immer „schlechter konditioniert“. Das CG-Verfahren braucht dann entweder viel mehr Iterationen oder scheitert daran, die letzten Nachkommastellen präzise zu berechnen.

3.7 Approximationsfehler LU vs CG

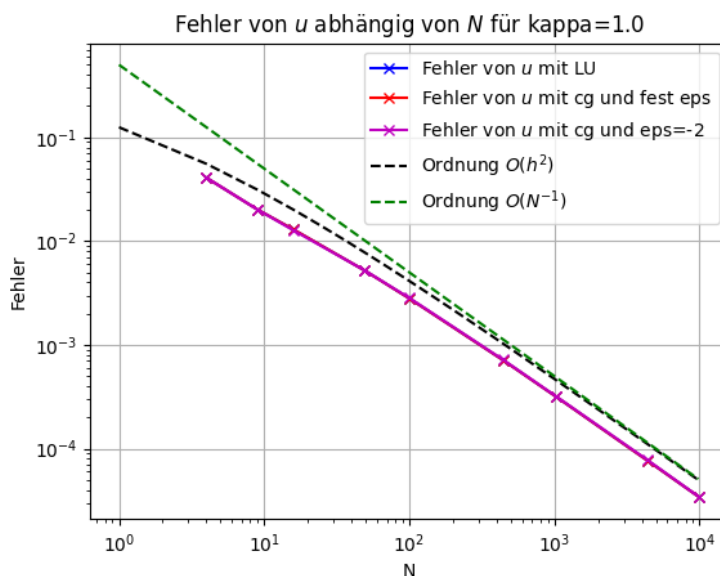


Abbildung 7: Fehler von u von dem CG und LU-Verfahren

Parameter: Bis $n = 100$, $\varepsilon_1 = 10^{-11}$, $\varepsilon_2 = h^4$

Beobachtung

Alle Grafen liegen übereinander und konvergieren mit $\mathcal{O}(N^{-1})$.

Abbruchkriterium: **eps**

Auswertung Es ist das selbe wie vorher, es gibt keinen Unterschied bei der Genauigkeit, sondern nur bis zu welchem n man kommt.

3.8 Laufzeit CG-Verfahren vs LU-Zerlegung

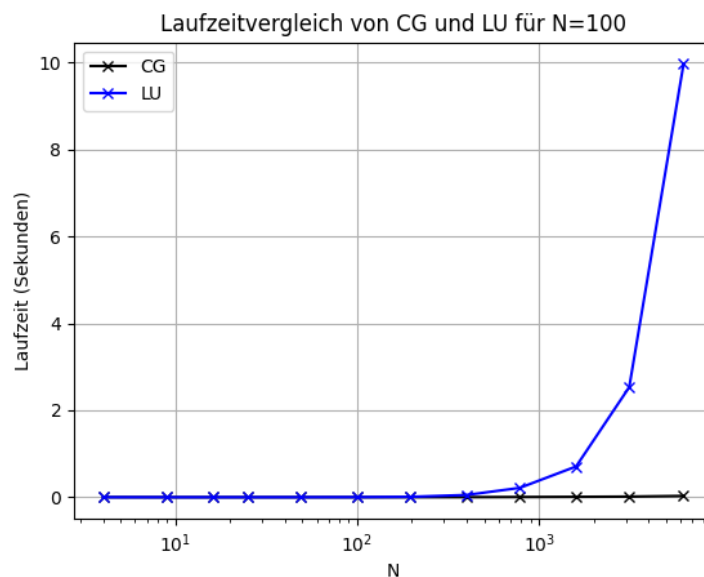


Abbildung 8: Laufzeit von LU vs CG

Parameter: Bis $n = 100$, $\varepsilon_1 = 10^{-11}$ und jeweils zehn Testdurchläufe.

Beobachtung

Der blaue Graf(LU) wächst wesentlich schneller als der schwarze(CG).

Auswertung

Diese Grafik verdeutlicht den Effizienzunterschied zwischen einem direkten Verfahren (LU-Zerlegung) und einem iterativen Verfahren (Verfahren der konjugierten Gradienten, CG) bei der numerischen Lösung der Poisson-Gleichung.

Wichtigste Erkenntnisse aus der Grafik:

Skalierbarkeit (Komplexität): LU-Zerlegung (blaue Linie): Zeigt einen steilen, nichtlinearen Anstieg der Laufzeit bei zunehmendem N . Selbst bei dünnbesetzten Matrizen (wie beim Poisson-Problem) tritt das Problem des „Fill-in“ (Auffüllung mit Nicht-Null-Elementen) auf, was die Berechnungen erheblich verlangsamt und

bei großen N eine enorme Speicherkapazität erfordert.

CG-Verfahren (schwarze Linie): Bleibt im Vergleich zur LU-Zerlegung praktisch „flach“. Die Laufzeit steigt extrem langsam an. Für dünnbesetzte, symmetrische und positiv definite Matrizen (wie sie im Gleichungssystem der Poisson-Gleichung vorliegen) sind iterative Verfahren um Größenordnungen effizienter.

Divergenzpunkt: Bei kleinen Werten von N (bis ca. $10^2 = 100$) arbeiten beide Methoden nahezu instantan. Ab $N \approx 500$ wird der Unterschied kritisch. Bei der Markierung von $N \approx 6000$ benötigt das LU-Verfahren etwa 10 Sekunden, während das CG-Verfahren lediglich Bruchteile einer Sekunde beansprucht. Warum ist CG bei diesem Problem besser?

Dünnbesetztheit: Die Matrix im Poisson-Problem enthält sehr viele Nullen. Das CG-Verfahren nutzt lediglich Matrix-Vektor-Multiplikationen, wodurch es extrem effizient mit dünnbesetzten Strukturen arbeiten kann, ohne unnötige Daten zu speichern. Problemspezifik: Die Poisson-Gleichung liefert eine symmetrische und positiv definite Matrix – dies sind die idealen Voraussetzungen für die Anwendung des CG-Verfahrens.

4 Zusammenfassung

- Optimale Fehlertoleranz: Unsere Experimente (siehe Abbildung 5) zeigen, dass eine Kopplung der Toleranz an die Schrittweite, etwa $\varepsilon = h^2$ oder $\varepsilon = h^4$, optimal ist. Eine noch kleinere Toleranz führt zwar zu mehr Iterationen, verbessert aber den totalen Fehler $e_u^{(n)}$ nicht weiter, da dieser dann durch den Diskretisierungsfehler der finiten Differenzen dominiert wird.
- Skalierbarkeit und Matrixgröße: Während die LU-Zerlegung bei feinen Gittern ($n > 100$) aufgrund des exponentiell steigenden Speicherbedarfs an Grenzen stößt, konnte das CG-Verfahren das Problem auch für deutlich größere N problemlos lösen.
- Konvergenzverhalten: Die lineare Konvergenz konnte experimentell bestätigt werden (Abbildung 6). Es zeigte sich zudem, dass die Konditionszahl $\kappa(A)$ mit sinkendem h (steigendem n) wächst, was die Anzahl der benötigten Iterationen moderat erhöht. Dennoch bleibt der Rechenaufwand pro Iteration aufgrund der sparse-Struktur minimal.
- Vergleich der Verfahren: Das LU-Verfahren ist bei sehr kleinen Systemen aufgrund der fehlenden Iterationsschleife schneller, verliert diesen Vorteil aber rapide mit steigender Dimension N . Für das Poisson-Problem in 2D und insbesondere 3D ist das CG-Verfahren aufgrund der Speicher- und Laufzeiteffizienz das überlegene Verfahren.
- Es zeigte sich, dass direkte Verfahren wie die LU-Zerlegung aufgrund von

Speicherbedarf und Füll-in-Effekten für große Systeme ungeeignet sind. Iterative Verfahren bieten hier deutliche Vorteile durch geringen Speicherbedarf und niedrigen Rechenaufwand pro Iteration.

- Die LU-Zerlegung ist dafür allgemein, man kann mit ihr alle linearen Gleichungssysteme lösen und mit dem CG-Verfahren nur wo symmetrische und positiv definite Matrizen auftreten, wie in unserem Fall. Deshalb sollte man die Verfahren abhängig von den Anforderungen wählen.
- Die Ergebnisse bestätigen die theoretischen Erwartungen und verdeutlichen die Bedeutung effizienter Abbruchkriterien sowie die Eignung iterativer Verfahren für große dünnbesetzte lineare Gleichungssysteme.

5 Python-Dokumentation

5.1 Overview

Dieses Projekt implementiert die numerische Verfahren zur Lösung der zweidimensionalen Poisson-Gleichung mit auf Null gesetzten Randbedingungen unter Verwendung von Finite-Differenzen. Es umfasst den Aufbau der Matrix A , eine spärliche Darstellung von A , die LU-Zerlegung, das CG-Verfahren, iterative Lösungsverfahren, die Fehlerberechnung sowie Visualisierungen. Es gibt fünf verschiedene Dateien:

- `experiments_it.py` – Hauptschnittstelle zur Durchführung von Experimenten
- `block_matrix_2d.py` – Aufbau und LU-Zerlegung der Matrix A der Linearisierung
- `poisson_problem_2d.py` – Berechnung der numerischen Lösung, Visualisierung und Fehlerberechnung
- `linear_solvers.py` – Löser für lineare Gleichungssysteme mit gegebener LU-Zerlegung
- `sparse_A_Matrix.py` – Analytische Speicherberechnung

5.2 `experiments_it.py`

5.2.1 Purpose

Es definiert eine exakte analytische Lösung sowie die dazugehörige rechte Seite und ermöglicht anschließend den Vergleich numerischer Lösungen (hauptsächlich mit der Konjugierten-Gradienten-Methode) mit der exakten Lösung. Das Programm unterstützt Fehleranalysen, Untersuchungen des Iterationsfehlers, Vergleiche zwischen LU- und CG-Verfahren sowie verschiedene Visualisierungsoptionen abhängig von benutzergewählten Parametern.

5.2.2 Functions

exact_u_func(x, $\kappa = 1$) **Input:** x (array of floats) stellt den Punkt im Raum dar, für den die Lösung ausgewertet wird

kappa (int) Parameter im Quellterm

Returns: (float) analytische Lösung der zweidimensionalen Poisson-Gleichung
analytische Lösung: $u(x) = x_1 x_2 \sin(\kappa \pi x_1) \sin(\kappa \pi x_2)$.

f_source(x, kappa=1) **Input:** x (array of floats) stellt den Punkt im Raum dar, für den die Lösung ausgewertet wird

kappa (int) Parameter im Quellterm

Returns: laplace (float) der Wert der Funktion $f(x)$
Rechte Seite berechnet aus $f = -\Delta u$.

main() Bietet verschiedene Experimentiermodi an:

- 1 Funktion u und die approximierte Lösung mit cg
- 2 Approximationsfehler für verschiedene N mit cg
- 3 Approximationsfehler für verschiedene N und unterschiedliche k mit cg
- 4 Approximationsfehler der Iteration
- 5 Approximationsfehler unterschiedlicher k
- 6 Approximationsfehler LU, cg
- 7 Approximationsfehler mit cg in Text
- 8 Approximationsfehler für verschiedene k mit cg in Text
- 9 Approximationsfehler für verschiedene k_{var} mit cg in Text
- 10 Approximationsfehler für nur ein n und verschiedene k_{var} mit cg in Text
- 11 Differenz von Funktion u und der approximierte Lösung mit cg
- 12 Laufzeitvergleich von LU-Zerlegung und cg
- 13 Laufzeitvergleich für verschiedene n von LU-Zerlegung und cg
- 0 Beenden

5.3 block_matrix_2d.py

5.3.1 Purpose

Es bietet Werkzeuge für die numerische lineare Algebra zur Analyse von Finite-Differenzen-Laplace-Operatoren, einschließlich der Bewertung von Speicherbedarf und Sparsität, LU-Zerlegung und der Erstellung des rechten Seitenvektors zur Lösung von Poisson-Gleichungen.

5.3.2 Class: BlockMatrix

Stellt eine Matrix mit der Gittergröße n und $N = (n - 1)^2$ Unbekannten dar.

Input: n (int) Anzahl der Intervalle in jeder Dimension

Raises: (Value Error) if $n < 2$

Konstruiert eine sparse Blockmatrix für den diskreten Laplace-Operator unter Verwendung des 5-Punkte-Schemas, mit Analysetools.

Attributes

- n (int) Anzahl der Intervalle in jeder Dimension
- N (int) Gesamtanzahl der inneren Gitterpunkte, berechnet als $(n - 1)^2$, verwendet zur Bestimmung der Gesamtgröße der Matrix
- A_sparse (scipy.sparse.csr_matrix) sparse Darstellung der Blockmatrix. Anfangs None, später berechnet
- L (numpy.ndarray) untere Dreiecks-Matrix mit Einheitsdiagonale von A
- U (numpy.ndarray) obere Dreiecks-Matrix von A
- P (numpy.ndarray) Permutationsmatrix aus der LU-Zerlegung von A

Alle oben genannten Attribute sind nicht statisch.

Methods

- `get_sparse()` – konstruiert eine sparse Matrix

Input: self (Blockmatrix object) enthaltend:

self. n (int) Anzahl der Intervalle in jeder Dimension

self. A_sparse (scipy.sparse.csr_matrix) zwischengespeicherte sparse Matrix

Returns: self. A_sparse (scipy.sparse.csr_matrix) sparse CSR-Matrix, die die Finite-Differenzen-Blockmatrix für den zweidimensionalen Laplace-Operator darstellt

- `eval_sparsity()` – Gibt die Anzahl der Nicht-Null-Elemente zurück

Input: self (Blockmatrix object) enthaltend:

self. N (int) Gesamtanzahl der inneren Gitterpunkte $(n - 1)^2$, verwendet zur Berechnung der Gesamtgröße der Matrix

self.get_sparse() (self. A_sparse (scipy.sparse.csr_matrix)) sparse Matrix, deren Sparsity bewertet wird

Returns: nnz (int) Anzahl der Nicht-Null-Elemente in der sparse Matrix
 rel_nnz (float) Verhältnis der Nicht-Null-Elemente zur Gesamtanzahl der möglichen Elemente in der vollständigen $N \times N$ Matrix

- `get_lu()` – LU-Zerlegung

Input: `self` (Blockmatrix object) enthaltend:

`self.A_sparse` (scipy.sparse.csr_matrix) zu faktorisierende sparse Matrix
`self.P` , `self.L` , `self.U` (numpy.ndarray) zwischengespeicherte Komponenten der LU-Zerlegung

Returns: `L` (numpy.ndarray) untere Dreiecks-Matrix mit Einheitsdiagonale von `A`

`U` (numpy.ndarray) obere Dreiecks-Matrix von `A`

`P` (numpy.ndarray) Permutationsmatrix aus der LU-Zerlegung von `A`

- `eval_sparsity_lu()` – Nicht-Null-Elemente in L und U

Input: `self` (Blockmatrix object) enthaltend:

`self.n` (int) Anzahl der Intervalle in jeder Dimension

`self.N` (int) Gesamtanzahl der inneren Gitterpunkte $(n-1)^2$, verwendet zur Berechnung der Gesamtgröße der Matrix

`self.get_lu` (`p`, `l`, `u`)

Returns: `nnz` (int) Anzahl der Nicht-Null-Elemente in der sparse Matrix

`rel_nnz` (float) Verhältnis der Nicht-Null-Elemente zur Gesamtanzahl der möglichen Elemente in der vollständigen $N \times N$ Matrix

- `show_nonzeros()` – Visualisierung der Sparsität

Input: `self` (Blockmatrix object) enthaltend:

`self.get_sparse` (`A` (scipy.sparse.csr_matrix)) sparse Matrix, deren Sparsitätsmuster geplottet wird

Returns: die Methode gibt keinen Wert zurück

Ausgabe ist eine Matplotlib-Abbildung, die das Nicht-Null-Muster der Matrix zeigt.

- `get_cond()` – Konditionszahl

Input: `self` (Blockmatrix object) enthaltend:

`self.get_sparse` (`A` (scipy.sparse.csr_matrix)) sparse Matrix, deren Konditionszahl berechnet wird.

Returns: `cond` (float) Konditionszahl der Matrix `A`

5.3.3 Additional Function

`plot_LU_A_N(n)` – Stellt die Skalierung der Sparsität für `A` und deren LU-Faktoren dar.

Input: n (int) Anzahl der Intervalle in jeder Dimension

Returns: Die Ausgabe ist ein Matplotlib-Diagramm, das die Sparsität von A , die Sparsität von $LU(A)$, sowie die Kurven $O(N)$ und $O(N^2)$ vergleicht.

rhs(n , f) – Berechnet den rechten Seitenvektor b für eine gegebene Funktion f .

Input: n (int) Anzahl der Intervalle in jeder Dimension

Returns: Gibt b als NumPy-Array der Länge $(n - 1)^2$ zurück, das dem rechten Seitenvektor des linearen Gleichungssystems $Au = b$ für die inneren Gitterpunkte entspricht.

Raises: (Value Error) falls $n < 2$

5.4 poisson_problem_2d.py

5.4.1 Purpose

Es enthält Funktionen zum Aufbau des linearen Gleichungssystems, zur Lösung mittels LU-Zerlegung oder der Conjugate-Gradient-Methode, zur Berechnung von Fehlern im Vergleich zur analytischen Lösung und zur Visualisierung der Ergebnisse mit 3D-Flächenplots und Fehlerdiagrammen. Zusätzlich unterstützt es die Analyse des Verhaltens der Solver bei unterschiedlichen Gittergrößen und CG-Parametern.

5.4.2 Funktionen

- **rhs**(n , f , $\kappa = 1$) – konstruiert den rechten Seitenvektor an den inneren Knoten

Input: n (int) Anzahl der Intervalle in jeder Dimension

f (callable) rechte-Seiten-Funktion des Poisson-Problems mit Signatur $f(x, \kappa)$

x (numpy.ndarray of floats) Punkt mit zwei Koordinaten

κ (int)

Returns: b (numpy.ndarray of floats) rechter Seitenvektor der Länge $(n - 1)^2$, jedes Element ist $f([j/n, k/n], \kappa)$ ausgewertet an den inneren Gitterpunkten

Raises: (ValueError) falls $n < 2$

- **idx**(nx , n) – ordnet einen Diskretisierungspunkt einer Gleichungsnummer zu

Input: nx (list of int) Koordinaten eines Diskretisierungspunkts multipliziert mit n

n (int) Anzahl der Intervalle in jeder Dimension

Returns: index (int) Nummer der entsprechenden Gleichung im Poisson-Problem

- `inv_idx(m, n)` – ordnet eine Gleichungsnummer den Koordinaten des Diskretisierungspunkts zu

Input: m (int) Nummer einer Gleichung im Poisson-Problem

n (int) Anzahl der Intervalle in jeder Dimension

Returns: [nx0, nx1] (list of int) Koordinaten des Diskretisierungspunkts, der der Gleichung m entspricht, multipliziert mit n

- `compute_error(n, û, u,  $\kappa = 1$ )` – berechnet den maximalen absoluten Fehler an den inneren Punkten

Input: n (int) Anzahl der Intervalle in jeder Dimension

û (numpy.ndarray) Finite-Differenzen-Approximation der Lösung

u (callable) exakte Lösung mit Signatur $u(x, \kappa)$

kappa (int) Parameter

Returns: max_error (float) maximaler absoluter Fehler von û im Vergleich zu u an allen inneren Punkten

- `plot_error(n, u, f, logscale=True,  $\kappa = 1$ , method='LU', k=None)` – plottet numerischen Fehler und Konditionszahl

Input: n (int) maximale Anzahl der Intervalle in jeder Dimension

u (callable) exakte Lösung $u(x, \kappa)$

f (callable) rechte-Seiten-Funktion $f(x, \kappa)$

logscale (bool, default=True) Achsen in logarithmischer Skala darstellen

kappa (int) Parameter in exakter Lösung und f

method (str, 'LU' oder 'cg') numerische Lösungsmethode

k (int, optional) Parameter für CG-Solver

Returns: matplotlib-Diagramm, das maximale Fehler (error_list), Konditionszahlen (condition_list) und Referenzkurven $O(N)$, $O(h^2)$, $O(N^{-1})$ zeigt

- `compute_u(n, f,  $\kappa = 1$ , method='LU', k=None)` – berechnet die Finite-Differenzen-Lösung des Poisson-Problems

Input: n (int) Anzahl der Intervalle in jeder Dimension

f (callable) rechte-Seiten-Funktion $f(x, \kappa)$

kappa (int) Parameter

method (str) Lösungsmethode ('LU' oder 'cg')

k (int, optional) CG-Solver-Parameter

Returns: `u` (numpy.ndarray) numerische Lösung an den inneren Punkten

- `plot_surface_comparison(n, u_func, f,  $\kappa = 1$ , method='LU', k=None)`
– 3D-Flächenplot numerische vs exakte Lösung

Input: `n` (int) Anzahl der Intervalle in jeder Dimension

`u_func` (callable) exakte Lösung $u(x, \kappa)$

`f` (callable) rechte-Seiten-Funktion $f(x, \kappa)$

`kappa` (int) Parameter

`method` (str) Lösungsmethode

`k` (int, optional) CG-Solver-Parameter

Returns: matplotlib 3D-Plot, der numerische Approximation und exakte Lösung vergleicht

- `compute_iter_error(n, u, f,  $\kappa = 1$ )` – berechnet und plottet Iterationsfehler für CG-Methode

Input: `n` (int) Anzahl der Intervalle in jeder Dimension

`u` (callable) exakte Lösung $u(x, \kappa)$

`f` (callable) rechte-Seiten-Funktion $f(x, \kappa)$

`kappa` (int) Parameter

Returns: matplotlib log-log-Plot der Iterationsfehler bei verschiedenen CG-Iterationen

- `plot_error_k(n, u, f, logscale=True,  $\kappa = 1$ )` – plottet Fehler für verschiedene CG-Parameter `k`

Input: `n` (int) maximale Anzahl der Intervalle in jeder Dimension

`u` (callable) exakte Lösung $u(x, \kappa)$

`f` (callable) rechte-Seiten-Funktion $f(x, \kappa)$

`logscale` (bool) Achsen in logarithmischer Skala darstellen

`kappa` (int) Parameter

Returns: matplotlib-Diagramm, das maximale Fehler für verschiedene CG `k`-Werte zeigt

- `plot_error_both(n, u, f, logscale=True,  $\kappa = 1$ )` – vergleicht LU- und CG-Fehler

Input: `n` (int) maximale Anzahl der Intervalle in jeder Dimension

`u` (callable) exakte Lösung $u(x, \kappa)$

`f` (callable) rechte-Seiten-Funktion $f(x, \kappa)$

`logscale` (bool) Achsen in logarithmischer Skala darstellen

kappa (int) Parameter

Returns: matplotlib-Diagramm, das Fehler des LU-Solvers und des CG-Solvers vergleicht, mit Referenzkurven $O(h^2)$ und $O(N^{-1})$

5.5 linear_solvers.py

5.5.1 Purpose

Der Code stellt Implementierungen von linearen Gleichungslösern bereit, einschließlich direkter LU-Zerlegung und des iterativen Konjugierten-Gradienten-Verfahrens. Er ermöglicht das Lösen sparse linearer Gleichungssysteme, die aus diskretisierten PDEs entstehen, verfolgt Iterationsverlauf und Residuen und unterstützt konfigurierbare Abbruchkriterien. Das Framework erleichtert numerische Experimente und die Analyse des Verhaltens von Lösern für unterschiedliche Systemgrößen und Parameter.

5.5.2 Implemented

`solve_lu(p, l, u, b)` – löst $Ax = b$ unter Verwendung der LU-Zerlegung.

Input: p (numpy.ndarray) Permutationsmatrix aus der LU-Zerlegung von A
l (numpy.ndarray) untere Dreiecksmatrix mit Einheitsdiagonale aus der LU-Zerlegung von A

u (numpy.ndarray) obere Dreiecksmatrix aus der LU-Zerlegung von A

b (numpy.ndarray of floats) rechte Seite des linearen Systems $Ax = b$

Returns: x (numpy.ndarray of floats) Lösung des linearen Systems $Ax = b$

`solve_cg(A, b, x0, params)` – löst $Ax = b$ mit der Methode des konjugierten Gradienten (CG).

Input: A (scipy.sparse.csr_matrix) Systemmatrix des linearen Gleichungssystems

b (numpy.ndarray of floats) rechte Seite des linearen Gleichungssystems

x0 (numpy.ndarray of floats) Anfangsschätzung der Lösung

params (dict, optional) Wörterbuch mit Abbruchbedingungen:

- eps : float, Toleranz für die Unendlich-Norm des Residuals
- max_iter : int, maximale Anzahl an Iterationen
- var_x : float, minimale Änderung der Iterierten in Unendlich-Norm

Returns: key (str) Grund für das Abbrechen (welches Abbruchkriterium erfüllt wurde)

x_list (list of numpy.ndarray) Iterierte der Lösung, beginnend mit x0

r_list (list of floats) Residuen jeder Iteration in der Unendlich-Norm

Raises: ValueError, falls keine Abbruchbedingung aktiv ist (d.h. $\text{eps} \leq 0$, $\text{max_iter} \leq 0$, $\text{var_x} \leq 0$)

5.5.3 Placeholders

`solve_sor(A, b, x0, params, omega)`, `solve_gs(A, b, x0, params)`, `solve_es(A, b, x0, params)` – Platzhalterfunktionen für SOR, Gauss–Seidel und Jacobi.

5.6 time_comp.py

5.6.1 Purpose

`plot_benchmark(n, func1, func2, runs=10)` – erstellt einen Laufzeitvergleichsplot zwischen zwei Funktionen

Input: `n` (int) – maximale Problemgröße (Anzahl der Intervalle / Gitterpunkte)

`func1` (callable) – erste Funktion zum Benchmarken

`func2` (callable) – zweite Funktion zum Benchmarken

`runs` (int, optional) – Anzahl der Durchläufe pro Funktion für Mittelwertbildung, Standard = 10

Returns: None – erzeugt ein matplotlib-Diagramm, das die durchschnittliche Laufzeit von `func1` und `func2` in Abhängigkeit von der Problemgröße zeigt. Die x-Achse wird logarithmisch dargestellt. Referenzkurven, z.B. $O(N)$, können hinzugefügt werden

`measure(func)` – interne Hilfsfunktion von `benchmark`, misst die durchschnittliche Laufzeit einer Funktion

Input: `func` (callable) – auszuführende Funktion, die einen Parameter akzeptiert

Returns: `avg_time` (float) – durchschnittliche Ausführungszeit in Sekunden über die definierte Anzahl von Durchläufen

5.7 User manual

```
def main():
    print("Wähle ein kappa")
    k = float(input("kappa = ").strip())

    print("1 - Funktion u und die approximierte Lösung mit cg "
          "(plot_surface_comparison)")
    print("2 - Approximationsfehler für verschiedene N mit cg "
          "(plot_error)")
    print("3 - Approximationsfehler für verschiedene N und unterschiedliche k mit cg "
          "(plot_error)")
    print("4 - Approximationsfehler der Iteration "
          "(compute_iter_error)")
    print("5 - Approximationsfehler unterschiedlicher k "
          "(plot_error_k)")
    print("6 - Approximationsfehler LU, cg "
          "(plot_error_both)")
    print("7 - Approximationsfehler mit cg in Text "
          "(plot_error_text)")
    print("8 - Approximationsfehler für verschiedene k mit cg in Text "
          "(plot_error_text)")
    print("9 - Approximationsfehler für verschiedene k_var mit cg in Text "
          "(plot_error_text)")
    print("10 - Approximationsfehler für nur ein n und verschiedene k_var mit cg in Text "
          "(plot_error_text_1n)")
    print("11 - Differenz von Funktion u und der approximierte Lösung mit cg "
          "(plot_surface_dif)")
    print("12 - Laufzeitvergleich von LU-Zerlegung und cg "
          "(benchmark)")
    print("13 - Laufzeitvergleich für verschiedene n von LU-Zerlegung und cg "
          "(plot_benchmark)")

    print("0      Beenden")

    choice = input("Your choice: ").strip()

    if choice == "1":
        print("\nFür welches n möchten Sie den Plot sehen?")
        n = int(input("n = ").strip())
        plot_surface_comparison(n, exact_u_func, f_source, kappa=k, method='cg')
    elif choice == "2":
        print("\nFür welches n möchten Sie den Plot sehen?")
        n = int(input("n = ").strip())
        plot_error(n, exact_u_func, f_source, kappa=k, method='cg')
    elif choice == "3":
        print("\nFür welches n möchten Sie den Plot sehen?")
        n = int(input("n = ").strip())
        print("\nFür welches k möchten Sie den Plot sehen?")
        k0 = int(input("k = ").strip())
        plot_error(n, exact_u_func, f_source, kappa=k, method='cg', k=k0)
    elif choice == "4":
        print("\nFür welches n möchten Sie den Plot sehen?")
        n = int(input("n = ").strip())
        compute_iter_error(n, exact_u_func, f_source, k)
    elif choice == "5":
        print("\nFür welches n möchten Sie den Plot sehen?")
        n = int(input("n = ").strip())
        plot_error_k(n, exact_u_func, f_source, kappa=k)
    elif choice == "6":
        print("\nFür welches n möchten Sie den Plot sehen?")
        n = int(input("n = ").strip())
        plot_error_both(n, exact_u_func, f_source, kappa=k)
    elif choice == "7":
        print("\nFür welches n möchten Sie den Plot sehen?")
        n = int(input("n = ").strip())
        plot_error_text(n, exact_u_func, f_source, kappa=k, method='cg')
```

```

elif choice == "8":
    print("\nFür welches n möchten Sie den Plot sehen?")
    n = int(input("n = ").strip())
    print("\nFür welches k möchten Sie den Plot sehen?")
    k0 = int(input("k = ").strip())
    plot_error_text(n, exact_u_func, f_source, kappa=k, method='cg', k=k0)
elif choice == "9":
    print("\nFür welches n möchten Sie den Plot sehen?")
    n = int(input("n = ").strip())
    print("\nFür welches k möchten Sie den Plot sehen?")
    k0 = int(input("k = ").strip())
    plot_error_text(n, exact_u_func, f_source, kappa=k, method='cg', k_var=k0)
elif choice == "10":
    print("\nFür welches n möchten Sie den Plot sehen?")
    n = int(input("n = ").strip())
    print("\nFür welches k möchten Sie den Plot sehen?")
    k0 = int(input("k = ").strip())
    plot_error_text_1n(n, exact_u_func, f_source, kappa=k, method='cg', k_var=k0)
elif choice == "11":
    print("\nFür welches n möchten Sie den Plot sehen?")
    n = int(input("n = ").strip())
    plot_surface_dif(n, exact_u_func, f_source, kappa=k, method='cg')
elif choice == "12":
    print("\nFür welches n möchten Sie die Laufzeit vergleichen?")
    n = int(input("n = ").strip())
    print("\nWie viel Mal soll es testen?")
    runs = int(input("runs = ").strip())
    avg1, avg2 = benchmark(cg, lu, n, runs)
    print(f"Durchschnittliche Laufzeit Funktion CG: {avg1:.8f} Sekunden")
    print(f"Durchschnittliche Laufzeit Funktion LU: {avg2:.8f} Sekunden")
elif choice == "13":
    print("\nFür welches n möchten Sie die Laufzeit vergleichen?")
    n = int(input("n = ").strip())
    print("\nWie viel Mal soll es testen?")
    runs = int(input("runs = ").strip())
    plot_benchmark(n, cg, lu, runs)

elif choice == "0":
    print("Exiting program.")
else:
    print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()

```

Wählen Sie aus, welches Diagramm angezeigt werden soll:

- 1 | Funktion u und die approximierte Lösung mit cg
 - 2 | Approximationsfehler für verschiedene N mit cg
 - 3 | Approximationsfehler für verschiedene N und unterschiedliche k mit cg
 - 4 | Approximationsfehler der Iteration
 - 5 | Approximationsfehler unterschiedlicher k
 - 6 | Approximationsfehler LU, cg
 - 7 | Approximationsfehler mit cg in Text
 - 8 | Approximationsfehler für verschiedene k mit cg in Text
 - 9 | Approximationsfehler für verschiedene k_var mit cg in Text
 - 10 | Approximationsfehler für nur ein n und verschiedene k_var mit cg in Text
 - 11 | Differenz von Funktion u und der approximierte Lösung mit cg
 - 12 | Laufzeitvergleich von LU-Zerlegung und cg
 - 13 | Laufzeitvergleich für verschiedene n von LU-Zerlegung und cg
 - 0 | Beenden
- Your choice:

Vergleich exakter und approximierter Lösung

Zeigt ein 3D-Oberflächendiagramm der exakten Lösung und der Approximation unter Verwendung des Conjugate Gradient (CG) Solvers. Es wird nach der Gittergröße n gefragt (z.B. 10 für 10 Intervalle pro Achse).

Approximationsfehler für verschiedene Gittergrößen

Zeigt ein Diagramm des Fehlers der numerischen Lösung in Abhängigkeit von der Gittergröße N .

Fehleranalyse für verschiedene Gittergrößen und k

Zeigt den Fehler für verschiedene k -Werte und Gittergrößen N .

Fehlerentwicklung während der Iteration

Zeigt, wie der Fehler im Verlauf der Iterationen abnimmt.

Fehler für verschiedene k -Werte

Zeigt, wie sich der Fehler bei unterschiedlichen k -Werten ändert.

Vergleich von LU- und CG-Fehlern

Zeigt die Fehler zweier verschiedener Solver (LU-Zerlegung und CG) im gleichen Diagramm.

Fehler in Textform (CG)

Gibt die Fehlerwerte für verschiedene Gittergrößen N in der Konsole aus.

Fehler in Textform für verschiedene k -Werte (CG)

Wie Option 7, jedoch für verschiedene k -Werte.

Fehler in Textform für verschiedene k_{var} (CG)

Gibt die Fehlerwerte in der Konsole für verschiedene Werte von k_{var} aus.

Fehler in Textform für ein festes n und verschiedene k_{var} (CG)

Zeigt die Fehlerwerte für ein festes n und mehrere k_{var} in der Konsole.

Differenz zwischen exakter und approximierter Lösung (CG)

Zeigt ein 3D-Oberflächendiagramm der Differenz zwischen exakter und numerischer Lösung.

Laufzeitvergleich von CG und LU

Misst und gibt die durchschnittliche Laufzeit von CG- und LU-Solvern für eine bestimmte Problemgröße aus.

Laufzeitvergleich für verschiedene N (CG vs LU)

Zeigt einen Plot der durchschnittlichen Laufzeiten von CG- und LU-Solvern über verschiedene Problemgrößen N .

Beenden

Beendet das Programm.

Eingaben im Detail

- **kappa:** Eine positive Zahl, z.B. 1.0. Beeinflusst die exakte Lösung.
- **n:** Gittergröße, d.h. Anzahl der Intervalle pro Achse.
Kleinere Werte $\rightarrow$ gröbere Approximation,
größere Werte $\rightarrow$ genauere Ergebnisse.

6 Quellenverzeichnis

Literatur

- [1] Hella Rabus, *Vorlesung Einführung in das wissenschaftliche Rechnen*, Sommersemester 2025
- [2] Caren Tischendorf, *Numerische Lineare Algebra, Skript zur Vorlesung im WiSe 2025/26*, WiSe 2025-2026
- [3] Otto Forster, *Analysis 1: Differential- und Integralrechnung einer Veränderlichen*, 11. Auflage, Springer Spektrum, 2013
- [4] Peter Deufelhard, Andreas Hohmann, *Numerische Mathematik 1: Eine algorithmisch orientierte Einführung*, 4. Auflage, de Gruyter, 2008
- [5] Fedir Deineko, Alexander Huhn, *Handout 1*, WiSe 2025-2026
- [6] Fedir Deineko, Alexander Huhn, *Handout 2*, WiSe 2025-2026
- [7] M.R. Hestenes, E. Stiefel, *Methods of conjugate gradients for solving linear systems. In: Journal of Research of the National Bureau of Standards*, 1952.

[8] <https://nhigham.com/2020/03/19/what-is-a-condition-number/>